# Kubernetes Essentials

## 1: Understanding Kubernetes Architecture and Core Components

### Introduction

Containerization has revolutionized how we build, ship, and run applications. While Docker provides the foundation for container management on a single host, and Docker Swarm offers basic orchestration capabilities, modern application deployments demand more robust solutions. This is where Kubernetes enters the picture.

Kubernetes (derived from the Greek word for "helmsman" or "pilot") is an open-source container orchestration platform that Google open-sourced in 2014. It has since grown into the industry standard for managing containerized workloads at scale.

### Why Kubernetes When We Already Have Docker Swarm?

If you have experience with Docker Swarm, you might wonder why Kubernetes is necessary. While Swarm is simpler to set up and works well for smaller deployments, Kubernetes offers:

- More sophisticated auto-healing mechanisms
- Fine-grained control over pod placement and resource allocation
- Built-in service discovery and load balancing
- Rolling updates and rollbacks with detailed control
- Horizontal auto-scaling based on custom metrics
- Extensibility through custom resources and operators
- A larger ecosystem of tools and cloud provider integrations

### Prerequisites for This Guide

Before diving into Kubernetes, ensure you have:

- Working knowledge of Linux command line
- Understanding of Docker concepts (images, containers, volumes, networks)
- Basic familiarity with Docker Swarm (services, stacks)
- MERN stack development experience (helpful for the application deployment examples)

### Setting Up Your Learning Environment on Ubuntu

For learning purposes, we will use Minikube, which creates a single-node Kubernetes cluster on your local machine. This simulates a multi-node environment without requiring multiple physical or virtual machines.

**Installing Minikube on Ubuntu**

Recommended to follow official docs.

- https://minikube.sigs.k8s.io/docs/start

```
# Update your system
sudo apt-get update && sudo apt-get upgrade -y

# Install required dependencies
sudo apt-get install -y apt-transport-https curl

# Download and install Minikube
curl -LO https://github.com/kubernetes/minikube/releases/latest/download/minikube-
linux-amd64
sudo install minikube-linux-amd64 /usr/local/bin/minikube

# Install kubectl (Kubernetes command-line tool)
# curl -LO "https://dl.k8s.io/release/$(curl -L -s
https://dl.k8s.io/release/stable.txt)/bin/linux/amd64/kubectl"
# sudo install -o root -g root -m 0755 kubectl /usr/local/bin/kubectl

# The kubectl required for minikube is part of it.
alias kubectl="minikube kubectl --"

# Start Minikube
minikube start --driver=docker

# Check minikube status
minikube status

# Verify the cluster is running
kubectl cluster-info
```

## Understanding Kubernetes Cluster Architecture

A Kubernetes cluster consists of two types of machines: master nodes (control plane) and worker nodes. When you deploy Kubernetes, you get a cluster that represents the entire orchestration environment.

### The Control Plane (Master Node Components)

The control plane is the brain of the Kubernetes cluster. It makes all decisions about what runs where, how scaling happens, and how the system maintains health. In a production environment, you would run multiple master nodes for high availability. For learning with Minikube, both control plane and worker functions run on a single node.

### kube-apiserver

The API server acts as the front door to the Kubernetes control plane. It exposes the Kubernetes API, which is used by all internal components, kubectl commands, and external clients. Every operation you perform, whether through the command line or programmatically, goes through the API server. It validates requests, processes them, and updates the etcd database accordingly. The API server is stateless, meaning you can run multiple instances for high availability.

### etcd

Etcd is a distributed, consistent key-value store that serves as Kubernetes' database. It stores all cluster data, including pod states, configurations, secrets, and node information. This is the source of truth for your entire cluster. If etcd is lost, your cluster loses its state, making it a critical component that requires regular backups.

**kube-scheduler**

The scheduler watches for newly created pods that have no assigned node. It then selects the most suitable node for each pod based on several factors:

- Resource requirements (CPU and memory requests)
- Node affinity and anti-affinity rules
- Taints and tolerations
- Pod priorities
- Custom scheduling policies

**kube-controller-manager**

This component runs controller loops that maintain the desired state of the cluster. Each controller watches the cluster state through the API server and makes changes to move the current state toward the desired state. Important controllers include:

- Node Controller: Manages node status and responds to node failures
- Replication Controller: Ensures the correct number of pod replicas are running
- Endpoint Controller: Manages service endpoint objects
- Service Account & Token Controllers: Create default accounts and API access tokens

**cloud-controller-manager (optional)**

When running Kubernetes on cloud providers like AWS, GCP, or Azure, this component manages the integration between Kubernetes and the cloud provider's services. It handles cloud-specific tasks such as provisioning load balancers, storage volumes, and managing node lifecycle.

**Worker Node Components**

Worker nodes run the actual containerized applications. Each worker node contains the following components:

**kubelet**

The kubelet is the primary agent that runs on every worker node. It communicates with the API server, ensures containers defined in PodSpecs are running and healthy, reports node and pod status back to the control plane, and watches for pod definitions assigned to its node. The kubelet then uses the container runtime to run those containers.

**kube-proxy**

Kube-proxy maintains network rules on each node. It ensures consistent networking across the cluster and implements the Kubernetes Service abstraction. Using either iptables or IPVS, kube-proxy forwards traffic to the correct pod endpoints and provides load balancing between pods behind a service.

**Container Runtime**

This is the actual software responsible for running containers. The kubelet interacts with the runtime through the Container Runtime Interface (CRI). Common runtimes include:

- containerd (default on most modern Kubernetes distributions)
- CRI-O
- Docker Engine (deprecated as of Kubernetes v1.24, though images can still be used)

For the latest Kubernetes versions, containerd is the recommended runtime.

## Understanding Cluster Configurations

### Single-Node Cluster (Minikube)

Minikube creates a fake cluster on a single machine, combining master and worker functions. This is suitable only for learning and development, not for production workloads.

### Multi-Node Single-Master Cluster

This configuration uses one master node and one or more worker nodes. It is appropriate for development, staging, and testing environments where high availability is not critical.

### Multi-Node Multi-Master Cluster (High Availability)

Also known as an HA cluster, this setup uses multiple master nodes and multiple worker nodes. It is required for pre-production (UAT) and production environments where cluster availability is paramount.

## Key Kubernetes Objects

Kubernetes organizes application components into several fundamental objects:

### Pod

A pod is the smallest and simplest unit in the Kubernetes object model. It represents a single instance of a running process in your cluster. A pod encapsulates one or more containers, storage resources, a unique network IP, and options that govern how containers should run. While pods can contain multiple containers (using patterns like sidecar or init containers), the most common pattern is one container per pod.

Init container Pattern (The Prep Work): Temporary helper that executes tasks during the pod's startup phase. It must run to completion and exit successfully before your main application container is allowed to start. Example: Cloning a GitHub repository or downloading configuration files into a shared volume.

Sidecar Pattern (The Constant Companion): A permanent helper that runs continuously throughout the entire lifecycle of the pod. It starts alongside your main application and stays alive until the main application stops. It enhances or extends the functionality of the main container without modifying the main application's code. Example: A tool that reads the log files generated by your main app and sends them to a central server.

### Service

A service is an abstraction that defines a logical set of pods and a policy for accessing them. Since pods are ephemeral and can be recreated with different IP addresses, services provide a stable endpoint for accessing pods.

Service types include:

- ClusterIP: Exposes the service on an internal cluster IP, making it reachable only from within the cluster
- NodePort: Exposes the service on each node's IP at a static port
- LoadBalancer: Exposes the service externally using a cloud provider's load balancer

**Volume**

Volumes provide storage that persists beyond the lifecycle of individual containers. Unlike Docker volumes, Kubernetes volumes have the same lifetime as the pod that encloses them.

**Namespace**

Namespaces provide a scope for names and a way to divide cluster resources between multiple users, teams, or projects. Resources must have unique names within a namespace but not across namespaces. Namespaces cannot be nested, and each resource can belong to only one namespace.

## Higher-Level Abstractions

Building upon the basic objects, Kubernetes provides higher-level abstractions that manage pods and replicas:

**ReplicaSet**

A ReplicaSet ensures that a specified number of pod replicas are running at any given time. If there are too many pods, it terminates the extras. If there are too few, it starts more pods. Unlike manually created pods, pods maintained by a ReplicaSet are automatically replaced if they fail, are deleted, or are terminated.

**Deployment**

A Deployment provides declarative updates for Pods and ReplicaSets. You describe a desired state in a Deployment, and the Deployment Controller changes the actual state to the desired state at a controlled rate. Deployments enable:

- Rolling out ReplicaSets
- Declaring new states for pods
- Rolling back to earlier deployment versions
- Scaling deployments up or down
- Cleaning up old ReplicaSets

## Understanding YAML for Kubernetes

Kubernetes uses YAML (YAML Ain't Markup Language) for configuration files. YAML is case-sensitive, does not allow tabs (spaces only), and uses .yaml or .yml file extensions.

YAML supports three data types:

- Scalars: Single values like numbers or strings (quotes are optional for strings)
- Maps: Key-value pairs (e.g., `name: person1`)
- Lists: Collections of values (e.g., `- item1`)

A basic pod definition in YAML looks like:

```
apiVersion: v1
kind: Pod
metadata:
  name: myapp-pod
  labels:
    app: myapp
spec:
  containers:
  - name: myapp-container
    image: httpd
```

---

# 2: Hands-On with Pods, Services, and Deployments

## Introduction

In Chapter 1, we explored the architectural foundation of Kubernetes—its control plane components, worker node responsibilities, and the core objects that form the building blocks of containerized applications. Now it is time to put that knowledge into practice. This chapter focuses on interacting with a live Kubernetes cluster using Minikube on Ubuntu. You will learn how to create and manage pods, expose them as network services, and use Deployments for declarative application management.

## Prerequisites for Chapter 2

Before proceeding, ensure you have:

- Minikube installed and running
- kubectl configured to communicate with your Minikube cluster
- Basic understanding of Docker images and containers
- Familiarity with YAML syntax

Verify your cluster status:

```
minikube status
kubectl cluster-info
kubectl get nodes
```

You should see a single node (minikube) in Ready state.

## Working with Namespaces

Namespaces provide a way to organize and isolate resources within a cluster. For learning purposes, you can use the default namespace, but understanding namespaces is essential for real-world deployments.

List existing namespaces:

```
kubectl get namespaces
```

Typical output includes default, kube-system, kube-public, and kube-node-lease. The kube-system namespace hosts control plane components. For our exercises, we will use the default namespace.

To create a new namespace for your learning projects:

```
kubectl create namespace learning
```

You can specify a namespace in subsequent commands using the `-n` flag.

## Creating Your First Pod

A pod is the smallest deployable unit in Kubernetes. Let us create a pod running the Apache HTTP server (httpd) using a declarative YAML file.

Quickstart:

```
kubectl get nodes

kubectl run my-httpd --image=httpd

kubectl get pods

kubectl describe pod my-httpd

kubectl logs my-httpd

kubectl exec -it my-httpd -- /bin/bash

kubectl port-forward pod/my-httpd 8086:80

http://localhost:8086

kubectl delete pod my-httpd
```

Standard practice is to create pods using YAML files. Create a file named `httpd-pod.yaml`:

```
apiVersion: v1
kind: Pod
metadata:
  name: httpd-pod
spec:
  containers:
```

```
    - name: httpd-container
      image: httpd:latest
```

Apply the configuration to your cluster:

```
kubectl apply -f httpd-pod.yaml
```

Check the status of the pod:

```
kubectl get pods
kubectl get pods -o wide
```

The pod will initially show as Pending while the container image is pulled, then transition to Running. Use `kubectl describe pod httpd-pod` to see detailed events.

To access the HTTP server, we need to forward a local port to the pod:

```
kubectl port-forward pod/httpd-pod 8080:80
```

Now open a browser or use curl: `curl http://localhost:8080`. You should see the default Apache page.

Clean up the pod when done:

```
kubectl delete pod httpd-pod
```

## Pod Lifecycle and Multi-Container Pods

Pods can contain multiple containers that share the same network namespace and storage volumes. Common patterns include sidecar containers (e.g., logging agents) and init containers (setup tasks).

Example of a multi-container pod YAML:

```
apiVersion: v1
kind: Pod
metadata:
  name: multi-container-pod

spec:
  containers:
  - name: main-app
    image: httpd
    ports:
    - containerPort: 80
```

```
  - name: sidecar-logger
    image: busybox
    command: ["/bin/sh", "-c"]
    args:
      - while true; do echo "sidecar running"; sleep 30; done
```

This example demonstrates two containers running in the same pod. However, for most beginners, single-container pods are sufficient.

To check the busybox container output.

```
kubectl logs multi-container-pod -c sidecar-logger
```

To access the shell in the container.

```
kubectl exec multi-container-pod -c sidecar-logger -it -- /bin/sh
```

## Exposing Pods with Services

Pods are ephemeral — they can be deleted and recreated with new IP addresses. A Service provides a stable endpoint (IP and DNS name) that abstracts the underlying pods.

**ClusterIP Service**

The default Service type, ClusterIP, makes the Service reachable only within the cluster.

Create a file `httpd-service.yaml`:

```
apiVersion: v1
kind: Service
metadata:
  name: httpd-service
spec:
  selector:
    app: httpd
  ports:
  - protocol: TCP
    port: 80
    targetPort: 80
  type: ClusterIP
```

Note that the selector must match the labels on your pods. First, recreate the httpd pod (or create a new one with label `app: httpd`):

```yaml
# httpd-pod-with-labels.yaml
apiVersion: v1
kind: Pod
metadata:
  name: httpd-pod
  labels:
    app: httpd
spec:
  containers:
  - name: httpd
    image: httpd
    ports:
    - containerPort: 80
```

Apply both:

```
kubectl apply -f httpd-pod-with-labels.yaml
kubectl apply -f httpd-service.yaml
```

Inspect the Service:

```
kubectl get svc
kubectl describe svc httpd-service
```

The Service gets a cluster-internal IP (e.g., 10.96.0.1). To test it from within the cluster, you can run a temporary pod:

```
kubectl run test-pod --image=busybox -it --rm --restart=Never -- sh
# Inside the container:
wget -qO- http://httpd-service
exit
```

**NodePort Service**

To access the Service from outside the cluster (useful for Minikube), use NodePort. Create the Service YAML `httpd-nodeport.yaml`:

```yaml
apiVersion: v1
kind: Service
metadata:
  name: httpd-nodeport
spec:
  selector:
```

```
    app: httpd
  ports:
  - protocol: TCP
    port: 80
    targetPort: 80
    nodePort: 30080
  type: NodePort
```

Apply and access via Minikube IP:

```
kubectl apply -f httpd-nodeport.yaml
minikube ip
curl http://<minikube-ip>:30080
```

## Working with Deployments

While you can create pods directly, Deployments are the recommended way to manage stateless applications. A Deployment manages a ReplicaSet, which in turn manages the pod replicas. Deployments support rolling updates, rollbacks, scaling, and self-healing.

Create a file `httpd-deployment.yaml`:

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: httpd-deployment
  labels:
    app: httpd
spec:
  replicas: 3
  selector:
    matchLabels:
      app: httpd
  template:
    metadata:
      labels:
        app: httpd
    spec:
      containers:
      - name: httpd
        image: httpd:latest
        ports:
        - containerPort: 80
        resources:
          requests:
            memory: "64Mi"
            cpu: "250m"
          limits:
```

```
            memory: "128Mi"
            cpu: "500m"
```

Apply the Deployment:

```
kubectl apply -f httpd-deployment.yaml
```

Observe the created resources:

```
kubectl get deployments
kubectl get replicasets
kubectl get pods
```

You will see three pods, each with a unique name derived from the Deployment and ReplicaSet.

You can create Service `httpd-deployment-nodeport-svc.yaml`

```
apiVersion: v1
kind: Service
metadata:
  name: httpd-deployment-nodeport-svc
spec:
  type: NodePort
  ports:
  - port: 80          # The standard port for other apps inside the cluster
    targetPort: 80    # The port Apache listens on inside the container
    nodePort: 32001   # Optional: Pin a specific external port (30000-32767)
  selector:
    app: httpd        # Tells the service to direct traffic to your 3 pods
```

Apply the service.

```
kubectl apply -f httpd-deployment-nodeport-svc.yaml
```

Then test it on host machine:

```
kubectl get svc
minikube ip
curl http://<minikube-ip>:32001
```

**Scaling a Deployment**

Scaling can be done declaratively (edit the YAML and re-apply) or imperatively:

```
kubectl scale deployment httpd-deployment --replicas=5
```

Check the pods again—two new pods will be created.

**Rolling Updates**

Update the image version (e.g., from httpd:latest to httpd:2.4):

```
kubectl set image deployment/httpd-deployment httpd=httpd:2.4
```

Watch the rollout status:

```
kubectl rollout status deployment/httpd-deployment
```

During the update, Kubernetes terminates old pods and creates new ones in a controlled manner (by default, 25% max surge and 25% max unavailable). Verify the update:

```
kubectl describe deployment httpd-deployment | grep Image
```

**Rolling Back**

If an update fails (e.g., a typo in image name), perform a rollback:

```
kubectl rollout undo deployment/httpd-deployment
```

View revision history:

```
kubectl rollout history deployment/httpd-deployment
```

## Deploying a MERN Full-Stack Application

Now we will deploy a simple MERN (MongoDB, Express.js, React, Node.js) application. The application consists of:

- A MongoDB database (using a StatefulSet or a simple Deployment with persistent volume, but for simplicity we use a Deployment with emptyDir for learning). You may use MySQL database as well.
- A backend Node.js/Express API

- A frontend React application served via Nginx or as static files

For brevity, we assume you have containerized images available on Docker Hub or a local registry. If not, you can build them locally and use Minikube's Docker daemon.

### Setting Up Minikube's Docker Environment

To use locally built images without pushing to a registry:

```
eval $(minikube docker-env)
```

Then build your images as usual with `docker build -t my-backend:latest .` inside that shell. The images become available to Minikube.

### MongoDB Deployment and Service

Create `mongodb-deployment.yaml`:

```yaml
apiVersion: v1
kind: Service
metadata:
  name: mongodb-service
spec:
  selector:
    app: mongodb
  ports:
  - port: 27017
    targetPort: 27017
  type: ClusterIP
---
apiVersion: apps/v1
kind: Deployment
metadata:
  name: mongodb-deployment
spec:
  replicas: 1
  selector:
    matchLabels:
      app: mongodb
  template:
    metadata:
      labels:
        app: mongodb
    spec:
      containers:
      - name: mongodb
        image: mongo:6.0
        ports:
        - containerPort: 27017
        env:
```

```
        - name: MONGO_INITDB_ROOT_USERNAME
          value: "admin"
        - name: MONGO_INITDB_ROOT_PASSWORD
          value: "password123"
```

Note: In production, use Secrets for credentials. For learning, this is acceptable.

**Backend (Node.js/Express) Deployment**

Assume your backend image is named `mern-backend:latest` and it connects to MongoDB using the service name `mongodb-service` as the host. It exposes port 5000.

Create `backend-deployment.yaml`:

```
apiVersion: v1
kind: Service
metadata:
  name: backend-service
spec:
  selector:
    app: backend
  ports:
  - port: 5000
    targetPort: 5000
  type: ClusterIP
---
apiVersion: apps/v1
kind: Deployment
metadata:
  name: backend-deployment
spec:
  replicas: 2
  selector:
    matchLabels:
      app: backend
  template:
    metadata:
      labels:
        app: backend
    spec:
      containers:
      - name: backend
        image: mern-backend:latest
        ports:
        - containerPort: 5000
        env:
        - name: MONGO_URI
          value: "mongodb://admin:password123@mongodb-service:27017/mydb?
authSource=admin"
        resources:
          requests:
```

```
            cpu: "100m"
            memory: "128Mi"
```

### Frontend (React) Deployment

For the frontend, build a production React app and serve it using Nginx. The Dockerfile might look like:

```
FROM node:18 AS build
WORKDIR /app
COPY package*.json ./
RUN npm install
COPY . .
RUN npm run build

FROM nginx:alpine
COPY --from=build /app/build /usr/share/nginx/html
EXPOSE 80
```

Build the image as `mern-frontend:latest` and then create `frontend-deployment.yaml`:

```
apiVersion: v1
kind: Service
metadata:
  name: frontend-service
spec:
  selector:
    app: frontend
  ports:
  - port: 80
    targetPort: 80
    nodePort: 30081
  type: NodePort
---
apiVersion: apps/v1
kind: Deployment
metadata:
  name: frontend-deployment
spec:
  replicas: 2
  selector:
    matchLabels:
      app: frontend
  template:
    metadata:
      labels:
        app: frontend
    spec:
      containers:
      - name: frontend
```

```
        image: mern-frontend:latest
        ports:
        - containerPort: 80
```

**Deploying the Full Stack**

Apply all YAML files in order:

```
kubectl apply -f mongodb-deployment.yaml
kubectl apply -f backend-deployment.yaml
kubectl apply -f frontend-deployment.yaml
```

Verify that all pods are running:

```
kubectl get pods
```

Check logs if any pod fails:

```
kubectl logs <backend-pod-name>
```

Access the frontend via Minikube IP and NodePort:

```
minikube ip
curl http://<minikube-ip>:30081
```

Or open your browser to that address. The frontend should communicate with the backend service (you may need to configure the React app's API base URL to point to the backend service's cluster IP or use an Ingress for cleaner routing).

## Cleaning Up Resources

To delete all resources created in this chapter:

```
kubectl delete deployment httpd-deployment mongodb-deployment backend-deployment
frontend-deployment
kubectl delete service httpd-service httpd-nodeport mongodb-service backend-
service frontend-service
kubectl delete pod httpd-pod multi-container-pod
```

If you created the learning namespace, delete it:

```
kubectl delete namespace learning
```

---

## Q. What is exact difference between pod and container?

The easiest way to understand the relationship is through an analogy: A Container is a single tenant, while a Pod is the shared apartment they live in. A container is the absolute lowest level where your application process actually runs. A Pod is a Kubernetes abstraction that acts as a wrapper around one or more of these containers. Here is exactly how they interact across all relevant technical aspects:

### 1. The Abstraction Layer

- The Container: Knows nothing about Kubernetes. It only knows about its own application code, environment variables, and local files.
- The Pod: Is the smallest atomic unit of deployment in Kubernetes. Kubernetes does not schedule individual containers onto machines; it schedules Pods. If a Pod moves to a new machine, all containers inside that Pod move together as a single unit.

### 2. Isolation vs. Sharing

Containers inside a Pod are deliberately semi-isolated from each other. They use Linux namespaces to share specific resources while keeping others completely locked down:

- Filesystem (Isolated by Default): Each container has its own isolated, read-only root filesystem. They cannot see each other's files unless you explicitly mount a shared Kubernetes Volume into both of them.
- Process Space (Isolated by Default): Container A cannot see or stop the processes running inside Container B. (Though you can optionally enable process sharing in the Pod settings if needed).

### 3. Network Aspect (Shared)

This is where the apartment analogy shines. Every container inside a single Pod shares the exact same network namespace, IP address, and MAC address.

- Localhost Communication: Because they share a network identity, Container A can talk to Container B instantly using localhost:port. For example, a frontend container can hit a backend sidecar at http://localhost:8080.
- Port Conflicts: Because they share the same IP, two containers inside the same Pod cannot listen on the same port. If Container A uses port 80, Container B will fail to start if it also tries to use port 80.
- External World: To the rest of the Kubernetes cluster, the Pod looks like a single machine with a single IP address.

### 4. Storage Aspect (Shared)

If containers inside a Pod need to share data (like a web server reading files generated by a content-downloader container), you define a Volume at the Pod level.

- The Pod owns the storage disk.
- You "mount" that same disk into specific folder paths inside each container.

## 5. Lifecycle and Scaling (Coupled)

- Co-scheduling: Containers in a Pod are tightly coupled. They are always created on the exact same physical or virtual machine (Node). You can never have Container A running on Node 1 and its Sidecar Container B running on Node 2.
- Fate Sharing: They live and die together. If the Pod is deleted or moved, all containers inside it are stopped and destroyed at the exact same moment.

---

## Summary Checklist

| Resource | Shared or Isolated? | What it means in practice |
|---|---|---|
| IP Address | Shared | All containers share one IP; talk via localhost. |
| Ports | Shared | Containers cannot use the same port number. |
| Storage | Shared (via Volumes) | Can read/write to the same folder if configured. |
| File System | Isolated | Individual system files are locked to each container. |
| Processes | Isolated | Containers cannot see each other's active processes. |